SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Brasilla R	**Reg. No.:**	192511357
Date:	28/10/2026	**Duration:**	60 minutes

Code of Conduct

I Brasilla R / 192511357 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: *Brasilla R*

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design**
   - Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization**
   - Create a Git repository for the project.
   - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow**
   - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
   - Explain the purpose of each Git operation performed.
6. **Commit History Analysis**
   - Present the commit history of the project.

    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
    - Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development**
    - Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.

Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
Problem Analysis & Project Design (20%)	/5
Git Repository & Branching Strategy(20%)	/4
Version Control Workflow & Commit History(10%)	/4
Docker Implementation(20%)	/4
Deployment, Testing & Results(15%)	/4
Documentation, Challenges & Future Enhancements(15%)	/4
TOTAL	/25

Git & Docker Technical Assignment

Vehicle Service and Maintenance Management System

Student: Brasilla R | Reg. No.: 192511357 | Course: CSA10 / CSA1063 Software Engineering

1. Problem Analysis

Assigned problem statement: Develop a system that allows vehicle owners to book service appointments, maintain service history, receive maintenance reminders, and track service status. The system is named the Vehicle Service and Maintenance Management System (VSMS).

Project objectives

- Replace phone calls and paper job cards with an online appointment booking facility.
- Keep a permanent, searchable service record for every registered vehicle.
- Remind owners automatically before the next service falls due.
- Give owners live visibility of the status of their vehicle at the service centre.
- Apply Git for version control and Docker for reproducible deployment.

Functional requirements

- **FR1 Vehicle registration:** an owner registers a vehicle with owner name, registration number and model.
- **FR2 Appointment booking:** an owner books a service type for a chosen date.
- **FR3 Service history:** completed services are stored and listed per vehicle.
- **FR4 Reminders:** vehicles whose next service is due within seven days are reported.
- **FR5 Status tracking:** status moves through Booked, In Progress, Completed.
- **FR6 Health check:** an endpoint reports whether the service is running.

Expected outcomes

A working three-tier application (web page, REST API, database) in which an owner can book a service, staff can update its status, and the system calculates the next due date (180 days after completion). The complete source is versioned in Git and runs identically on any machine with one command, *docker compose up*.

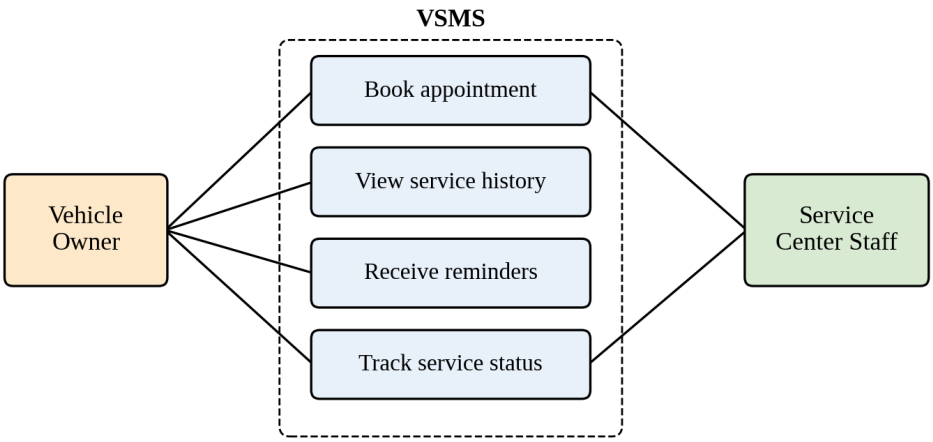


Figure 1: Use-case diagram of VSMS

2. Project Structure Design

The repository separates the backend, frontend and deployment files so that each part can be developed, tested and containerized independently.

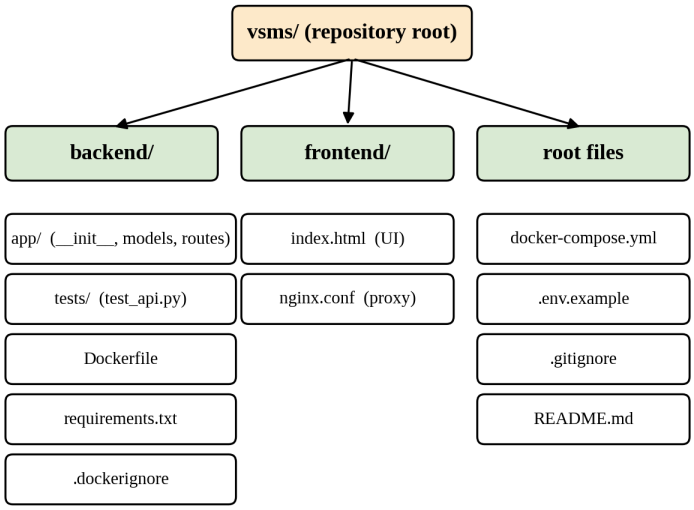


Figure 2: Repository folder structure

Purpose of major folders and files

- **backend/app/:** Flask application. *__init__.py* creates the app and database; *models.py* defines Vehicle and Appointment tables; *routes.py* holds the REST endpoints.
- **backend/tests/:** automated pytest cases that protect existing behaviour.
- **backend/Dockerfile and .dockerignore:** build recipe of the API image and the files excluded from it.
- **backend/requirements.txt:** pinned Python dependencies for repeatable builds.
- **frontend/index.html:** simple user interface that calls the API.
- **frontend/nginx.conf:** serves the page and proxies /api requests to the backend.
- **docker-compose.yml:** defines and connects all containers.
- **.env.example:** template of configuration values; the real .env is never committed.
- **.gitignore:** keeps cache files, secrets and local databases out of Git.
- **README.md:** project description and run instructions.

3. Git Repository Initialization

The repository was created and the first commit was made with the following steps.

```
mkdir vsms && cd vsms
git init -b main
git config user.name "Brasilla R"
git add .
git commit -m "chore: initialize repository with README, .gitignore and .env.example"
git branch develop
```

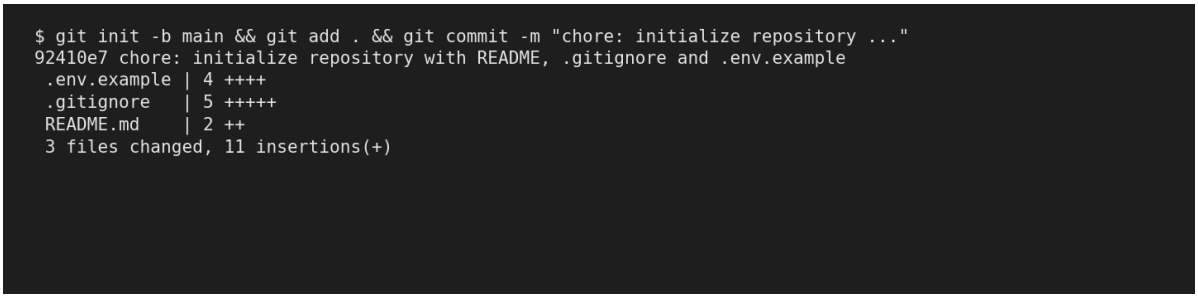


Figure 3: Git initialization and first commit

Purpose of the essential Git files

- **.git/ directory:** created by git init; stores all commits, branches and configuration.
- **.gitignore:** lists patterns (__pycache__, .env, venv, *.sqlite3) that Git must not track.
- **README.md:** first document a collaborator reads.
- **.env.example:** shows required variables without exposing real passwords.
- **.dockerignore:** the Docker equivalent of .gitignore for build context.

4. Git Branching Strategy

A **Git Flow** model is used with five branch types. Figure 4 shows how work moves between them.

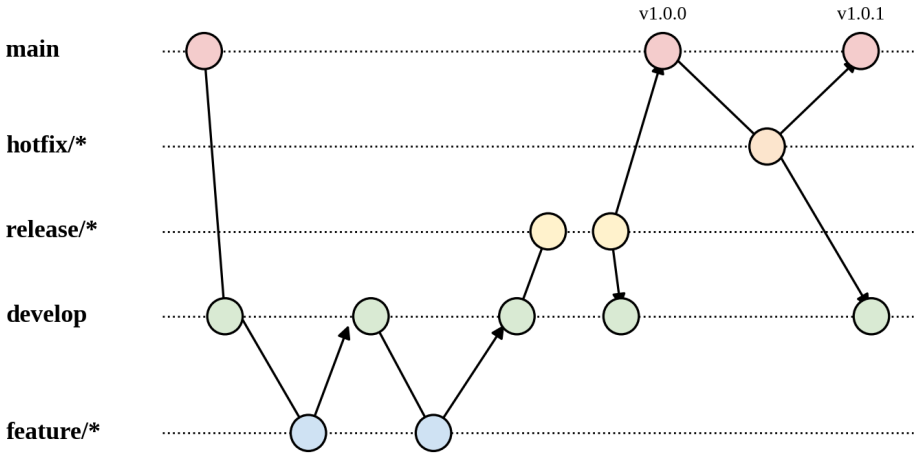


Figure 4: Branching strategy diagram

- **main:** always holds stable, released code; every release is tagged (v1.0.0, v1.0.1).
- **develop:** integration branch where completed features are combined and tested.
- **feature/*:** one branch per requirement, such as feature/appointment-booking, feature/service-history, feature/status-reminders and feature/dockerization.
- **release/*:** final preparation of a version (release/v1.0.0) before merging into main.
- **hotfix/*:** urgent fix created from main (hotfix/reminder-null-check) and merged into both main and develop.

Justification

VSMS has several independent modules (booking, history, reminders, containers) that can be built in parallel by different team members. Feature branches isolate unfinished work, develop gives a safe place to test combinations, and main remains deployable. Because a service centre depends on accurate reminders, the hotfix branch allows a production bug to be corrected without waiting for the next feature release. Tags give each deployed version a permanent name.

5. Version Control Workflow

The workflow demonstrated in the repository consists of the following operations.

```
git checkout -b develop
git checkout -b feature/appointment-booking
git add . && git commit -m "feat(api): add vehicle registration and
appointment booking endpoints"
git checkout develop
git merge --no-ff feature/appointment-booking
git remote add origin ../remote.git
git push -u origin --all
git push origin --tags
git pull origin develop
```

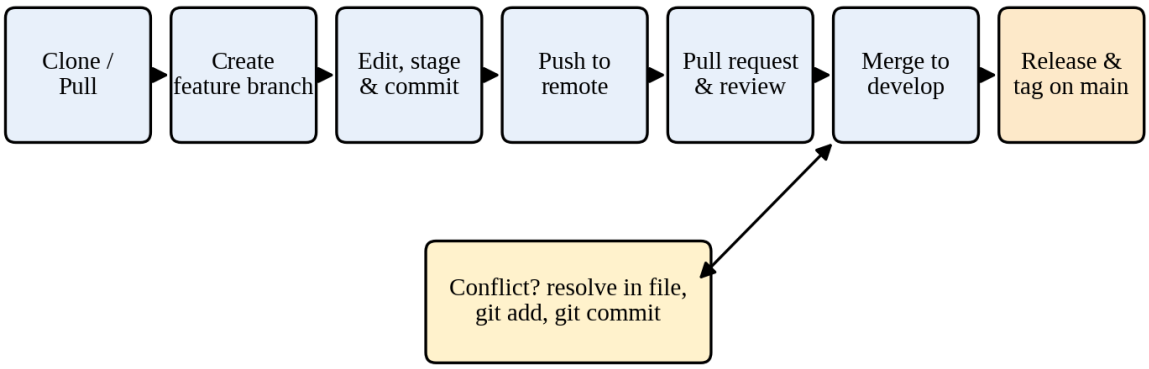


Figure 5: Git workflow diagram

Purpose of each operation

- **git checkout -b:** creates and switches to a new branch so work stays isolated.
- **git add / git commit:** stages changes and records a meaningful snapshot with a message.
- **git merge --no-ff:** joins a feature into develop and keeps a visible merge commit in history.
- **git remote add / git push:** links the local repository to a shared remote and uploads branches and tags.
- **git pull:** downloads and merges teammates' work so the local copy stays synchronized.
- **git tag -a:** marks release points.

Conflict resolution

Two branches, feature/service-history and feature/status-reminders, both appended new endpoints at the end of routes.py. Merging the second branch produced a conflict. The conflict markers were removed, both functions were kept, the file was staged and the merge was committed. Tests were then run to confirm that nothing was broken.

```
$ git merge feature/status-reminders
Auto-merging backend/app/routes.py
CONFLICT (content): Merge conflict in backend/app/routes.py
Automatic merge failed; fix conflicts and then commit the result.

$ git status --short
UU backend/app/routes.py

# after editing the file (keeping both functions):
$ git add backend/app/routes.py
$ git commit -m "merge: feature/status-reminders into develop (resolve routes.py conflict)"
```

Figure 6: Merge conflict and its resolution

6. Commit History Analysis

The complete history of all branches is shown below; the develop branch contains 15 commits, and the repository has two release tags and eight branches.

```
$ git log --oneline --graph --all --decorate
*   0e47596 (tag: v1.0.1, origin/main, main) hotfix: reminder null check
|\
| * 23ba84a (HEAD -> develop, origin/hotfix/reminder-null-check, origin/develop, hotfix/reminder-null-check) fix(api): guard reminders query against null next_due dates
|/
*   77ed8c8 (tag: v1.0.0) release: v1.0.0
|\
| * 4be6f36 (origin/release/v1.0.0, release/v1.0.0) docs: update README for v1.0.0 release
| *   24d9b07 merge: feature/dockerization into develop
| |\
| | * 4c8bbf0 (origin/feature/dockerization, feature/dockerization) build(docker): add docker-compose with db, backend and frontend services
| | * 249b78b build(docker): add backend Dockerfile, dockerignore, nginx frontend config
| |/
| * 248a8bd test(api): add pytest cases for health and booking-to-history flow
| *   e2deb4d merge: feature/status-reminders into develop (resolve routes.py conflict)
| |\
| | * d673f18 (origin/feature/status-reminders, feature/status-reminders) feat(api): add status tracking and maintenance reminder endpoints
| * |   d06dce1 merge: feature/service-history into develop
| |\ \
| | |/
| |/|
| | * fc0bfa2 (origin/feature/service-history, feature/service-history) feat(api): add service history endpoint
| |/
| *   f57d514 merge: feature/appointment-booking into develop
| |\
| | * c3a03ce (origin/feature/appointment-booking, feature/appointment-booking) feat(api): add vehicle registration and appointment booking endpoints
| |/
| * b476d52 feat(backend): add Flask app factory and Vehicle/Appointment models
|/
* 92410e7 chore: initialize repository with README, .gitignore and .env.example
```

Figure 7: git log --oneline --graph --all --decorate



Figure 8: Tags, branches and remote

Good commit practices followed

- **Conventional prefixes:** feat, fix, test, build, docs, chore and merge tell the reader the type of change at a glance.
- **Short imperative subject:** for example "add service history endpoint" states exactly what changed.
- **Small, single-purpose commits:** each commit can be reviewed, reverted or cherry-picked on its own.
- **Scope in brackets:** (api), (backend) and (docker) show which part of the project was affected.
- **Maintenance value:** the history acts as a changelog; git bisect and git revert can locate and undo faulty changes quickly.

7. Docker Environment Design

Why Docker is suitable

- **Consistency:** the same image runs on the developer laptop, the test server and production, which removes "works on my machine" problems.
- **Multi-service setup:** the Flask API, PostgreSQL and Nginx start together with one command.
- **Isolation:** each service has its own filesystem and dependencies.
- **Quick onboarding:** a new team member needs only Git and Docker.

- **Scalability:** more backend containers can be added when many owners book at the same time.

Components requiring containerization

- **Backend:** Flask REST API served by Gunicorn, built from our own Dockerfile.
- **Database:** PostgreSQL 16 using the official image with a persistent volume.
- **Frontend / reverse proxy:** Nginx serving the static page and forwarding /api calls.

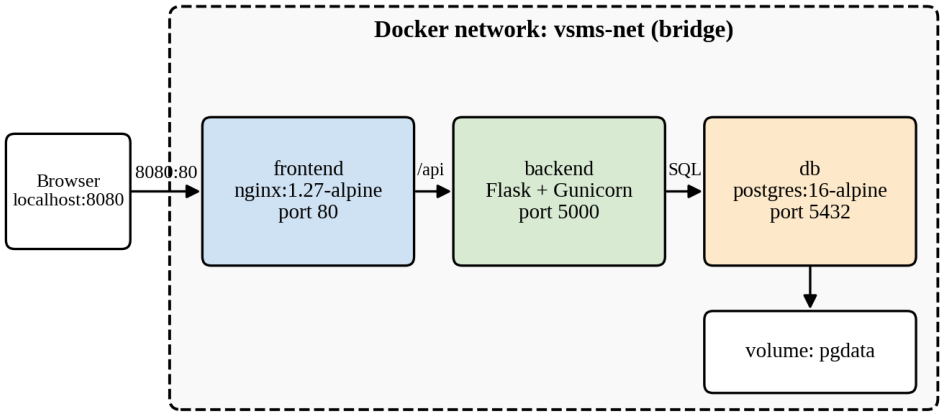


Figure 9: Docker architecture of VSMS

8. Dockerfile Development

The backend image is described by the following Dockerfile (backend/Dockerfile).

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd -m appuser && chown -R appuser /app
```

```
USER appuser
EXPOSE 5000
HEALTHCHECK CMD python -c "import urllib.request as
u;u.urlopen('http://localhost:5000/api/health')"
CMD ["gunicorn", "-b", "0.0.0.0:5000", "app:create_app()"]
```

Purpose of each instruction

- **FROM python:3.12-slim:** small official Python base image.
- **ENV:** prevents .pyc files and makes logs appear immediately.
- **WORKDIR /app:** sets the working directory inside the container.
- **COPY requirements.txt and RUN pip install:** installed before the code is copied so Docker caches the dependency layer and rebuilds faster.
- **COPY . .:** copies the application source.
- **RUN useradd / USER appuser:** runs the app as a non-root user for security.
- **EXPOSE 5000:** documents the port used by the API.
- **HEALTHCHECK:** lets Docker mark the container healthy only if /api/health responds.
- **CMD:** starts Gunicorn, a production-grade server, instead of the Flask development server.

9. Docker Compose Design

```
services:
  db:
    image: postgres:16-alpine
    env_file: .env
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks: [vsms-net]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER}"]
      interval: 5s
      retries: 5
  backend:
    build: ./backend
    env_file: .env
    depends_on:
      db:
        condition: service_healthy
    networks: [vsms-net]
  frontend:
    image: nginx:1.27-alpine
    ports:
      - "8080:80"
    volumes:
      - ./frontend/index.html:/usr/share/nginx/html/index.html:ro
      - ./frontend/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on: [backend]
    networks: [vsms-net]
volumes:
  pgdata:
```

```
networks:
  vsms-net:
```

- **Services:** db (PostgreSQL), backend (Flask API built from ./backend) and frontend (Nginx).
- **Networking:** all services join the bridge network vsms-net, so the backend reaches the database by the name *db* and Nginx reaches the API by the name *backend*. Only port 8080 is published to the host.
- **Volumes:** the named volume pgdata keeps database records after containers are removed; read-only bind mounts supply the frontend files.
- **Environment variables:** POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD and DATABASE_URL are read from the .env file, so no password is stored in the YAML or in Git.
- **Dependencies:** backend waits until the database health check passes (service_healthy); frontend starts after backend.

10. Container Deployment and Testing

Deployment steps

```
git clone <repository-url> && cd vsms
cp .env.example .env
docker compose up --build -d
docker compose ps
curl http://localhost:8080/api/health
```

```
$ docker compose up --build -d
[+] Running 4/4
 Network vsms_vsms-net   Created
 Container vsms-db-1        Healthy
 Container vsms-backend-1   Started
 Container vsms-frontend-1  Started

$ docker compose ps
NAME              IMAGE              STATUS                PORTS
vsms-db-1         postgres:16-alpine  Up (healthy)          5432/tcp
vsms-backend-1    vsms-backend        Up (healthy)          5000/tcp
vsms-frontend-1   nginx:1.27-alpine   Up                    0.0.0.0:8080->80/tcp

$ curl http://localhost:8080/api/health
{"status":"ok"}
```

Figure 10: Docker Compose execution

Testing procedure

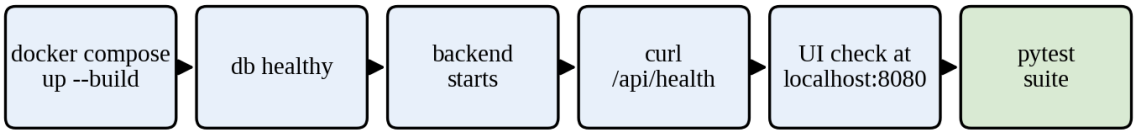


Figure 11: Testing procedure

- **Container check:** docker compose ps must show db and backend as healthy and frontend as running.
- **API check:** the health endpoint returns {"status":"ok"}.
- **Functional check:** a vehicle is registered, an appointment is booked, its status is changed to Completed, and the history endpoint returns that service.
- **Automated tests:** pytest verifies the same flow; both tests passed (Figure 12).
- **Persistence check:** after docker compose down and up again, the data is still present because of the pgdata volume.

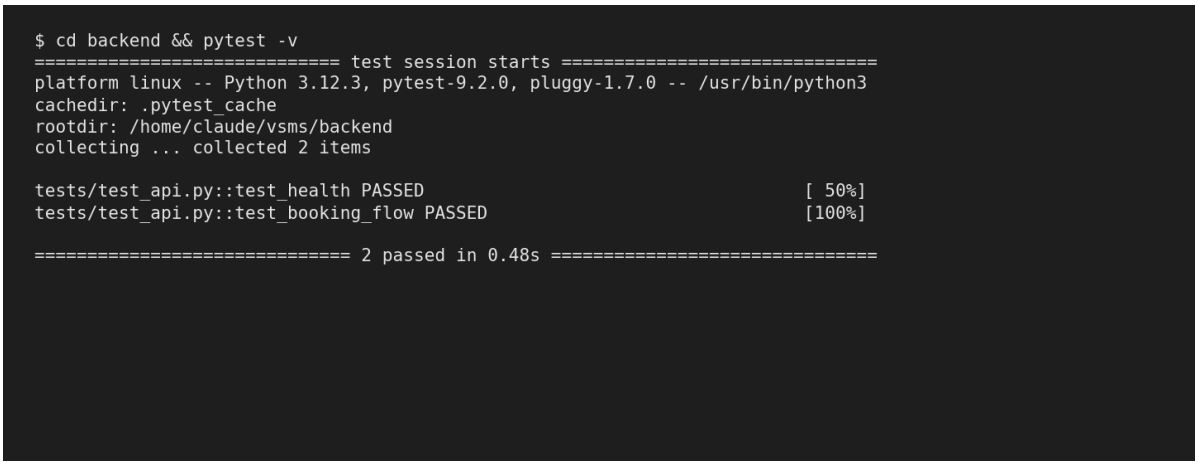


Figure 12: Automated test results

The application was therefore built, started and verified successfully through containers, and the evidence is shown in Figures 10 and 12.

11. Benefits of Git and Docker

- **Collaboration:** each member works on a separate feature branch; pull requests and code review keep quality high.
- **Version management:** every change to the booking, history and reminder modules is recorded, can be compared and can be rolled back; tags identify released versions.
- **Portability:** the application runs the same on Windows, Linux and macOS because the environment is packaged in images.
- **Deployment:** one command starts the whole system, and release tags can be mapped to image versions.
- **Scalability:** additional backend containers can be run behind Nginx as the number of customers grows.
- **Maintainability:** readable commit history, pinned dependencies, automated tests and small containers make fixing and upgrading easier.

12. Challenges and Improvements

Challenges encountered

- Resolving the merge conflict in routes.py when two features edited the same file.
- Starting order: the backend failed if PostgreSQL was not yet ready, which was solved by a database health check with depends_on.
- Keeping secrets such as database passwords out of Git, solved with .env and .env.example.
- Reducing image size and build time through a slim base image and layer caching.

Possible improvements

- **Git:** protect main and develop with mandatory pull-request reviews, add a CI pipeline (GitHub Actions) that runs pytest on every push, and use signed commits.
- **Docker:** use multi-stage builds, scan images for vulnerabilities, publish versioned images to a registry, and add Docker secrets instead of a plain .env file.
- **Deployment:** move to Kubernetes for auto-scaling and rolling updates, and add Nginx HTTPS.
- **Features:** SMS or e-mail reminder delivery, owner login, payment and invoice generation, and a mobile application.

References

- Chacon, S. and Straub, B., *Pro Git*, 2nd ed., Apress. https://git-scm.com/book
- Git Documentation. https://git-scm.com/doc
- Docker Documentation. https://docs.docker.com
- Docker Compose file reference. https://docs.docker.com/compose/
- Flask Documentation. https://flask.palletsprojects.com
- Conventional Commits Specification. https://www.conventionalcommits.org